

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:		Reg. No.:	
Section / Batch:		Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

Code of Conduct

I S.Mahammed Nayeem/ 192425294 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

S.Mahammed Nayeem

RUBRICS FOR CALCULATION

Coding Challenge Task

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm	25%				
Code Implementation	20%				
Competitive Performance	20%				

Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

AIM

To find the maximum length chain of pairs where the end value of one pair is less than the start value of the next pair using a Greedy approach in Python.

PROCEDURE

1. Read the number of pairs `n`.
2. Read the starting and ending values of each pair.
3. Sort all pairs based on their ending value.
4. Initialize `chain_end` to negative infinity and `count` to 0.
5. Traverse the sorted pairs.
6. If the start value of the current pair is greater than `chain_end`, select the pair.
7. Update `chain_end` with the current pair's end value and increment `count`.
8. Print `count`, which represents the maximum chain length.

PYTHON CODE

```
import sys
def maxChainLen(pairs):
    # Sort pairs based on ending value
    pairs.sort(key=lambda x: x[1])

    chain_end = float("-inf")
    count = 0

    for a, b in pairs:
        if a > chain_end:
            count += 1
            chain_end = b

    return count

def main():
    data = sys.stdin.read().strip().split()

    if not data:
        return

    idx = 0
```

```
n = int(data[idx])
idx += 1

pairs = []

for _ in range(n):
    a = int(data[idx])
    b = int(data[idx + 1])
    idx += 2
    pairs.append((a, b))

result = maxChainLen(pairs)
print(result)

if __name__ == "__main__":
    main()
```

INPUT

3 5 24 39 60 15 28

The pairs are:

(5, 24)
(39, 60)
(15, 28)

OUTPUT

2

Language: [Python](#) [C](#) [C++](#) [Java](#)

QUESTIONS

Q1

Maximum Length Chain of Pairs
100 marks

1/1 answered

Q1 Maximum Length Chain of Pairs

100 marks

Given pairs of numbers where each pair has a start and end value, the goal is to form the longest chain such that the end of one pair is less than the start of the next. Efficient selection is required to maximize chain length. Use Greedy or Dynamic Programming techniques.

Input Format:

n = number of pairs

pairs[][]

Your Code

```
1 import sys
2
3 def maxChainLen(pairs):
4     pairs.sort(key=lambda x: x[1])
5     chain_end = float("-inf")
6     count = 0
7     for a, b in pairs:
8         if a > chain_end:
9             count += 1
10            chain_end = b
11    return count
12
13 def main():
14     data = sys.stdin.read().strip().split()
15     if not data:
16         return
17     idx = 0
18     n = int(data[idx])
19     idx += 1
20
21     pairs = []
22     for _ in range(n):
23         a = int(data[idx]); b = int(data[idx + 1])
24         idx += 2
25         pairs.append([a, b])
26
27     print(maxChainLen(pairs))
28
29 if __name__ == "__main__":
30     main()
```

Input

3 5 24 39 60 15 28

Output

2

Submitted